

Name :- Atharva Sachin Jagtap
PRN :- 202501100043
Division :- SE1

Theory Assignment-02 : Numpy and pandas : Data Analysis

Dataset (same for all PS)

```
check.py X
check.py > ...
2  import pandas as pd
3  import numpy as np
4
5  data = {
6      'OrderID': [101,102,103,104,105,106,107,108,109,110],
7      'Region': ['North','South','East','West','North','South','East','West','North','South'],
8      'Product': ['A','B','C','A','B','C','A','B','C','A'],
9      'Sales': [200,150,300,250,400,350,500,450,600,550],
10     'Quantity': [2,3,4,2,5,3,6,4,7,5],
11     'Profit': [20,15,40,25,60,35,80,45,100,55]
12 }
13
14 df = pd.DataFrame(data)
15
```

1. Identify the top 3 orders with highest sales and display their complete details.

```
print("\n3. Top 3 Sales Orders")
top3 = df.sort_values(by='Sales', ascending=False).head(3)
print(top3)
```

OUTPUT:

```
3. Top 3 Sales Orders
  OrderID Region Product  Sales  Quantity  Profit
8      109  North      C    600         7     100
9      110  South      A    550         5      55
6      107   East      A    500         6      80
PS C:\Programming(CSE)\Python>
```

2. Compute the total sales of all orders and determine the average sales per order.

```
print("\n1. Total & Average Sales")
total_sales = df['Sales'].sum()
avg_sales = df['Sales'].mean()
print({"Total": total_sales, "Average": avg_sales})
```

OUTPUT:

```
PS C:\Programming(CSE)\Python> & c:\python314\python.exe "c:/Programming(CSE)/Python/check.py"

1. Total & Average Sales
{'Total': np.int64(3750), 'Average': np.float64(375.0)}
PS C:\Programming(CSE)\Python>
```

3. Compute the **profit margin (Profit/Sales)** for each order and identify the **order with the highest margin**.

```
print("\n4. Profit Margin & Highest")
df['Profit_Margin'] = df['Profit'] / df['Sales']
print(df.loc[df['Profit_Margin'].idxmax()])
```

OUTPUT:

```
4. Profit Margin & Highest
OrderID      109
Region       North
Product       C
Sales         600
Quantity       7
Profit        100
Profit_Margin 0.166667
Name: 8, dtype: object
PS C:\Programming(CSE)\Python>
```

4. Calculate the **total sales for each region** and display the results in **descending order**.

```
print("\n2. Region-wise Total Sales (Sorted)")
region_sales = df.groupby('Region')['Sales'].sum()
print(region_sales.sort_values(ascending=False))
```

OUTPUT:

```
2. Region-wise Total Sales (Sorted)
Region
North    1200
South    1050
East      800
West      700
Name: Sales, dtype: int64
PS C:\Programming(CSE)\Python>
```

5. Create a new column for **discount (10% of sales)** and compute the **final revenue after discount**.

```
print("\n8. Discount & Final Revenue")
df['Discount'] = df['Sales'] * 0.1
df['Final_Revenue'] = df['Sales'] - df['Discount']
print(df[['Sales', 'Discount', 'Final_Revenue']])
```

OUTPUT:

```
8. Discount & Final Revenue
   Sales  Discount  Final_Revenue
0    200     20.0         180.0
1    150     15.0         135.0
2    300     30.0         270.0
3    250     25.0         225.0
4    400     40.0         360.0
5    350     35.0         315.0
6    500     50.0         450.0
7    450     45.0         405.0
8    600     60.0         540.0
9    550     55.0         495.0
PS C:\Programming(CSE)\Python>
```

6. Normalize the Sales column using Min-Max scaling (NumPy) and add it as a new column.

```
print("\n10. Normalized Sales (NumPy)")
sales = np.array(df['Sales'])
normalized = (sales - sales.min()) / (sales.max() - sales.min())
df['Normalized_Sales'] = normalized
print(df[['Sales', 'Normalized_Sales']])
```

OUTPUT:

```
10. Normalized Sales (NumPy)
   Sales  Normalized_Sales
0     200         0.111111
1     150         0.000000
2     300         0.333333
3     250         0.222222
4     400         0.555556
5     350         0.444444
6     500         0.777778
7     450         0.666667
8     600         1.000000
9     550         0.888889
PS C:\Programming(CSE)\Python>
```

7. Generate a correlation matrix for Sales, Profit, and Quantity, and extract the correlation between Sales and Profit.

```
print("\n7. Sales-Profit Correlation")
corr = df[['Sales', 'Profit', 'Quantity']].corr()
print(corr)
print("Sales-Profit:", corr.loc['Sales', 'Profit'])
```

OUTPUT:

```
7. Sales-Profit Correlation
   Sales  Profit  Quantity
Sales  1.000000  0.900377  0.871506
Profit  0.900377  1.000000  0.958600
Quantity 0.871506  0.958600  1.000000
Sales-Profit: 0.9003767048731384
PS C:\Programming(CSE)\Python>
```

8. Calculate the total quantity sold per region and also determine the average quantity per region.

```
print("\n5. Total & Avg Quantity per Region")
grouped = df.groupby('Region')['Quantity']
print(pd.DataFrame({'Total': grouped.sum(), 'Average': grouped.mean()}))
```

OUTPUT:

```
5. Total & Avg Quantity per Region
   Total  Average
Region
East     10  5.000000
North    14  4.666667
South    11  3.666667
West      6  3.000000
PS C:\Programming(CSE)\Python>
```

9. Determine the **total quantity sold for each product** and identify the **most sold product**.

```
print("\n9. Most Sold Product (Quantity)")
product_qty = df.groupby('Product')['Quantity'].sum()
print(product_qty)
print("Top Product:", product_qty.idxmax())
```

OUTPUT:

```
9. Most Sold Product (Quantity)
Product
A    15
B    12
C    14
Name: Quantity, dtype: int64
Top Product: A
PS C:\Programming(CSE)\Python>
```

10. Filter all orders where **sales are greater than 300**, and compute the **total profit of those orders**.

```
print("\n6. Orders with Sales > 300 and Total Profit")
filtered = df[df['Sales'] > 300]
print(filtered)
print("Total Profit:", filtered['Profit'].sum())
```

OUTPUT:

```
6. Orders with Sales > 300 and Total Profit
  OrderID  Region  Product  Sales  Quantity  Profit
4      105   North      B    400         5     60
5      106   South      C    350         3     35
6      107   East      A    500         6     80
7      108   West      B    450         4     45
8      109   North      C    600         7    100
9      110   South      A    550         5     55
Total Profit: 375
```

11. Sort the dataset based on **quantity in ascending order** and extract the **bottom 3 records**.

```
print("\n13. Bottom 3 by Quantity")
print(df.sort_values(by='Quantity').head(3))
```

OUTPUT:

```
13. Bottom 3 by Quantity
  OrderID  Region  Product  Sales  Quantity  Profit
0      101   North      A    200         2     20
3      104   West      A    250         2     25
1      102   South      B    150         3     15
PS C:\Programming(CSE)\Python>
```

12. Calculate the average profit for each region and identify the region with the highest average profit.

```
print("\n11. Region with Highest Avg Profit")
region_profit = df.groupby('Region')['Profit'].mean()
print(region_profit)
print("Top Region:", region_profit.idxmax())
```

OUTPUT:

```
11. Region with Highest Avg Profit
Region
East      60.0
North     60.0
South     35.0
West      35.0
Name: Profit, dtype: float64
Top Region: East
PS C:\Programming(CSE)\Python>
```

13. Determine all orders where profit is greater than the average profit, and display their details.

```
print("\n15. Orders with Profit > Average")
avg_profit = df['Profit'].mean()
print(df[df['Profit'] > avg_profit][['OrderID', 'Profit']])
```

OUTPUT:

```
15. Orders with Profit > Average
  OrderID  Profit
4       105      60
6       107      80
8       109     100
9       110      55
PS C:\Programming(CSE)\Python>
```

14. Count the number of orders per product and compute the percentage contribution of each product.

```
print("\n12. Product Count & Percentage")
counts = df['Product'].value_counts()
percentage = (counts / len(df)) * 100
print(pd.DataFrame({'Count': counts, 'Percentage': percentage}))
```

OUTPUT:

```
12. Product Count & Percentage
  Count  Percentage
Product
A         4       40.0
B         3       30.0
C         3       30.0
PS C:\Programming(CSE)\Python>
```

15. Replace product labels (e.g., A → Prod_A, B → Prod_B) and determine the **number of unique products**.

```
print("\n16. Replace Product Labels & Unique Count")
df['Product'] = df['Product'].replace({'A': 'Prod_A', 'B': 'Prod_B'})
print(df['Product'].unique())
print("Unique Count:", df['Product'].nunique())
```

OUTPUT:

```
16. Replace Product Labels & Unique Count
<StringArray>
['Prod_A', 'Prod_B', 'C']
Length: 3, dtype: str
Unique Count: 3
PS C:\Programming(CSE)\Python>
```

16. Compute the **cumulative sales** based on order sequence and display the results.

```
print("\n14. Cumulative Sales")
df_sorted = df.sort_values(by='OrderID')
df_sorted['Cumulative_Sales'] = df_sorted['Sales'].cumsum()
print(df_sorted[['OrderID', 'Cumulative_Sales']])
```

OUTPUT:

```
14. Cumulative Sales
OrderID  Cumulative_Sales
0       101              200
1       102              350
2       103              650
3       104              900
4       105             1300
5       106             1650
6       107             2150
7       108             2600
8       109             3200
9       110             3750
PS C:\Programming(CSE)\Python>
```

17. Calculate the **percentage contribution** of each region to the total sales.

```
print("\n19. Region Contribution (%)")
total = df['Sales'].sum()
region_sales = df.groupby('Region')['Sales'].sum()
print((region_sales / total) * 100)
```

OUTPUT:

```
19. Region Contribution (%)
Region
East    21.333333
North   32.000000
South   28.000000
West    18.666667
Name: Sales, dtype: float64
PS C:\Programming(CSE)\Python>
```

18. Calculate the variance and standard deviation of the Sales column using NumPy.

```
print("\n17. Variance & Std Dev of Sales")
print("Variance:", np.var(df['Sales']))
print("Std Dev:", np.std(df['Sales']))
```

OUTPUT:

```
17. Variance & Std Dev of Sales
Variance: 20625.0
Std Dev: 143.61406616345073
PS C:\Programming(CSE)\Python>
```

19. Detect outliers in the Sales column using the condition ($\text{mean} \pm 2 \times \text{standard deviation}$).

```
print("\n20. Outliers in Sales")
mean = np.mean(df['Sales'])
std = np.std(df['Sales'])
outliers = df[(df['Sales'] > mean + 2*std) | (df['Sales'] < mean - 2*std)]
print(outliers)
```

OUTPUT:

```
20. Outliers in Sales
Empty DataFrame
Columns: [OrderID, Region, Product, Sales, Quantity, Profit]
Index: []
```

20. Group the dataset by Region and Product, and compute the total sales for each combination.

```
print("\n18. Region & Product Wise Sales")
multi = df.groupby(['Region', 'Product'])['Sales'].sum().reset_index()
print(multi)
```

OUTPUT:

```
18. Region & Product Wise Sales
  Region Product  Sales
0   East      A    500
1   East      C    300
2  North      A    200
3  North      B    400
4  North      C    600
5  South      A    550
6  South      B    150
7  South      C    350
8   West      A    250
9   West      B    450
PS C:\Programming(CSE)\Python>
```

THANK YOU!